Pairing Mentors and Mentees using Data Analysis and Algorithm

By Zach Liu

For Industrial Engineering Mentorship Program

Data Cleaning

```
#importing neccessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


# read in the tables
mentee = pd.read_csv('mentees.csv')
mentor = pd.read_csv('mentors.csv')


mentee.head()
```

Show hidden output

```
mentor.head()
```

Show hidden output

```
#looking at these data, need to clean the data by getting rid of the irrelevant columns

mentee.columns = mentee.columns.str.strip()  # Removes leading/trailing spaces
menteeCol = ['Timestamp', 'What program are you in?', 'Can we add you to a WhatsApp group with the other mentors and mentees?', 'Can we shar

mentor.columns = mentor.columns.str.strip()
mentorCol = ['Timestamp', 'Year', 'Why do you want to be a mentor?', 'Can we add you to a WhatsApp group with the other mentors and mentees?

mentee = mentee.drop(menteeCol, axis=1)
mentor = mentor.drop(mentorCol, axis=1)
mentee.head()
```

Show hidden output

```
mentor.head()
```

Show hidden output

```
# We know that there are null numbers, however, they don't affect too much.
mentee.isnull().sum()
```

Show hidden output

```
mentor.isnull().sum()
```

Show hidden output

```
# The column names are too complicated
len(mentor.columns)
len(mentee.columns)

mentee.columns = ['Mentee Email', 'Mentee Name', 'Phone Number', 'Mentee Hobbies', 'Minors and Certificates',
                  'Mentee Career Interests', 'Mentee Clubs', 'Mentee Preference', 'Mentee Communcation Method',
                  'Mentee Ethnicity', 'Mentee Language', 'Mentee Gender', 'Mentee Gender Preference']

mentor.columns = ['Mentor Email', 'Mentor Name', 'Phone Number', 'Mentor Hobbies', 'Minors and Certificates',
                  'Mentor Career Interests', 'Mentor Clubs', 'Mentor Preference', 'Mentor Communcation Method',
                  'Mentor Ethnicity', 'Mentor Language', 'Mentor Gender', 'Mentor Gender Preference']

# Check for duplicate names
```

```
print(mentor.duplicated(subset=['Mentor Name']).sum())
print(mentee.duplicated(subset=['Mentee Name']).sum())

# Drop duplicates
mentor = mentor.drop_duplicates(subset=['Mentor Name'])
mentee = mentee.drop_duplicates(subset=['Mentee Name'])
```

Show hidden output

```
mentee.head()
```

Show hidden output

```
mentor.head()
```

Show hidden output

Analysis

I am going to be using a weighted system to pair mentors and mentees. Different features will be weighted differently based on how important it is. One challenge is that some people have indicated specfic preferences when they filled out the form, so we would want to prioritze that the most. Their responses are, however, not organized. So I am thinking to use NLP tools to proceed the analysis with that. *Update, I also normalized other columns just to make my analysis easier.

```
# Importing necessary libraries
import nltk
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Downloading NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')


# Define the text processing function
def process_text(text):
  tokens = word_tokenize(text)
  stop_words = set(stopwords.words('english'))
  tokens = {word.lower() for word in tokens if word.isalnum() and word not in stop_words}
  lemmatizer = WordNetLemmatizer()
  tokens = {lemmatizer.lemmatize(word) for word in tokens}
  return tokens

# List of important columns containing texts to fill NA values and apply text processing
mentor_columns = [
    'Mentor Preference', 'Mentor Hobbies', 'Minors and Certificates',
    'Mentor Career Interests', 'Mentor Clubs', 'Mentor Ethnicity',
    'Mentor Language', 'Mentor Gender'
]

mentee_columns = [
    'Mentee Preference', 'Mentee Hobbies', 'Minors and Certificates',
    'Mentee Career Interests', 'Mentee Clubs', 'Mentee Ethnicity',
    'Mentee Language', 'Mentee Gender'
]

# Fill NA values and process text for mentor columns
for col in mentor_columns:
  mentor[col].fillna('', inplace=True)
  mentor[col] = mentor[col].astype(str).apply(process_text)

# Fill NA values and process text for mentee columns
for col in mentee_columns:
  mentee[col].fillna('', inplace=True)
  mentee[col] = mentee[col].astype(str).apply(process_text)

# Inspect the processed columns
mentor.reset_index(drop=True, inplace=True)
mentor
```

Show hidden output

```
mentee.reset_index(drop=True, inplace=True)
mentee
```

Show hidden output

Algorithm

```
# Now that we have converted and lemmatized the column, we can proceed to match them with categories to see which one needs to be emphasized
# To do this, we can use dictionary
categories = {
              'language': {'language', 'mandarin', 'chinese', 'english', 'speak'},
              'career': {'career', 'data', 'ai', 'artificial intelligence', 'software', 'trader', 'robotic'},
              'ethnicity': {'ethnicity', 'indonesian', 'muslim'}
              }

# Now we have manually sorted out the key words and categories that we need to pay attention to, it's time to start the pairing algorithm
# To start, I realized that in general it's the best to match people with similar interests and hobbies, which can be done after we normalize
# However, there are certain constraints that I must follow, for example, if Gender Preference is yes, then need to find someone with the sam
# Their written preferences will weigh more to be prioritzed. And depending on what catogory it is, we need to search for different columns t
# Since there are less mentees than mentors and mentees are priortized in this program, we need to pair mentee with mentor

# This takes in specific mentee and specific mentor, since we are gonna iterate through rows one by one
def calculate_score (mentee, mentor):
  score = 0

  # Checking if they priortize a certain feature, in other words, do they have preference in one of the categories above
  if (len(set(mentee['Mentee Preference']).intersection(categories['language'])) > 0) or (len(set(mentee['Mentee Preference']).intersection(c

    # Now we need to check which category it belongs exactly and attribute more weight to the feature accordingly
    if len(set(mentee['Mentee Preference']).intersection(categories['language'])) > 0:
      score += len(set(mentee['Mentee Language']).intersection(mentor['Mentor Language'])) * 10

    if len(set(mentee['Mentee Preference']).intersection(categories['career'])) > 0:
      score += len(set(mentee['Mentee Career Interests']).intersection(mentor['Mentor Career Interests'])) * 10

    if len(set(mentee['Mentee Preference']).intersection(categories['ethnicity'])) > 0:
      score += len(set(mentee['Mentee Ethnicity']).intersection(mentor['Mentor Ethnicity'])) * 10

  # If no specfic preference or no categories are detected
  score += len(set(mentee['Mentee Hobbies']).intersection(mentor['Mentor Hobbies'])) # Score from hobbies
  score += len(set(mentee['Minors and Certificates']).intersection(mentor['Minors and Certificates'])) # Score from minors and certificates
  score += len(set(mentee['Mentee Clubs']).intersection(mentor['Mentor Clubs'])) * 2 # Score from clubs
  score += len(set(mentee['Mentee Career Interests'].intersection(mentor['Mentor Career Interests']))) * 2 # Score from career interests
  score += len(set(mentee['Mentee Ethnicity'].intersection(mentor['Mentor Ethnicity']))) * 2 # Score from ethnicity
  score += len(set(mentee['Mentee Language'].intersection(mentor['Mentor Language']))) * 2 # Score from language

  return score

# If someone prefers the same gender then we pair them with the same gender
def check_gender (mentee, mentor):
  if mentee['Mentee Gender Preference'] == 'Yes':
    if mentee['Mentee Gender'] == mentor['Mentor Gender']:
      return True
    else:
      return False
  else:
    return True

# Now we have the calculate_score function and the checking_gender function, we can write the process
paired = set() # This is to prevent duplicate mentors
results = [] # This is to store the results

# Iterate through mentees and mentors
for _, mentees in mentee.iterrows(): #_ drops the index
  best_score = 0
  best_pair = None

  # Iterate through mentors
  for _, mentors in mentor.iterrows():

    mentor_name = mentors['Mentor Name']
```

```
        # Check if the mentor is already paired
        if mentor_name in paired:
          continue

        # Check if gender criteria meets first
        if check_gender(mentees, mentors):

          # Calculate the score
          score = calculate_score(mentees, mentors)
          if score > best_score:
            best_score = score
            best_pair = (mentees, mentors)

        # Skip to next person if gender doesn't match
        else:
          continue

      if best_pair is not None:
        paired.add(best_pair[1]['Mentor Name'])
        results.append({
          'Mentee Name': best_pair[0]['Mentee Name'],
          'Mentor Name': best_pair[1]['Mentor Name'],
          'Score': best_score
        })
    '''
    # Since we have more mentors than mentees, we have some extra mentors that don't have assigned mentees
    # Now we prioritize the mentors and assign them to different mentees

    # Initialize remaining mentors after the first round of pairing
    remaining_mentors = set(mentor['Mentor Name'] for _, mentor in mentor.iterrows()) - paired
    remaining_mentors_df = mentor[mentor['Mentor Name'].isin(remaining_mentors)]

    # Dictionary to track how many times each mentee has been paired, initialize to 0
    mentee_pair_count = {mentee['Mentee Name']: 0 for _, mentee in mentee.iterrows()}

    # Update mentee_pair_count based on initial results
    for result in results:
      mentee_pair_count[result['Mentee Name']] += 1

    # Iterate through remaining mentors
    for _, mentors in remaining_mentors_df.iterrows():
      best_score = 0
      best_pair = None

      for _, mentees in mentee.iterrows():

        # Check if the mentee has been paired fewer than 2 times
        if mentee_pair_count[mentees['Mentee Name']] < 2:

          # Check if gender criteria are met
          if check_gender(mentees, mentors):

            # Calculate the score
            score = calculate_score(mentees, mentors)

            if score > best_score:
              best_score = score
              best_pair = (mentees, mentors)

        # After finding the best mentee for the current mentor
      if best_pair is not None:
        mentees, mentors = best_pair
        paired.add(mentors['Mentor Name'])
        mentee_pair_count[mentees['Mentee Name']] += 1  # Increment the pairing count for the mentee
        results.append({
            'Mentee Name': mentees['Mentee Name'],
            'Mentor Name': mentors['Mentor Name'],
            'Score': best_score
        })

    '''
    # Convert the results to a DataFrame and save or print them
    results_df = pd.DataFrame(results)
    results_df = results_df.sort_values(by='Score', ascending=False)
    print(len(results_df["Mentee Name"].unique()))
    results_df.reset_index(drop=True, inplace=True)
```

```
results_df.to_excel('results.xlsx', index=False)
results_df
```

Show hidden output